# UVM

**Rama Krishna Vyata**
ramakrishna.vyata@smachines.com
**Sahara DV Team**

**05-May-2011 SM Silicon**
**04-Aug-2011 Sahara DV**
**20-Dec-2011 wpu-all**

For Internal Knowledge Sharing Purpose only.

# Confidential

## SM Silicon India Pvt Ltd

# Agenda

1. World of UVM
2. Brief understanding of Migration from HDL to UVM
3. Phases
4. Framework understanding (walk through a simple code)
5. Sample Make file and run an example

# 1. World of UVM

**What is UVM?**

- UVM is a methodology for functional verification using SystemVerilog, complete with a supporting library of SystemVerilog code. The letters UVM stand for the **Universal Verification Methodology.**

- UVM was created by Accellera based on the OVM (Open Verification Methodology) version 2.1.1. The roots of these methodologies lie in the application of the languages IEEE 1800™ SystemVerilog, IEEE 1666™ SystemC, and IEEE 1647™ e.

- UVM is explicitly simulation-oriented, but UVM can also be used alongside assertion-based verification, hardware acceleration or emulation.

- UVM test benches are more than **traditional HDL test benches**, which might wiggle a few pins on the design-under-test (DUT) and rely on the designer to inspect a waveform diagram to verify correct operation.

- UVM test benches are complete *verification environments* composed of reusable verification components, and used as part of an overarching methodology of constrained random, coverage-driven, verification.

# [Just a quick snippet] When its traditional test, what we remember

```
module test(PAddr, PWrite, PSel, PRData, Rst, clk);
// Port declarations omitted...

initial begin
// Drive reset
Rst <= 0;
#100 Rst <= 1;

// Drive Control bus
@(posedge clk)
PAddr <= 16’h50;
PWData <= 32’h50;
PWrite <= 1'b1;
PSel <= 1'b1;

// Toggle PEnable
@(posedge clk)
PEnable <= 1'b1;
@(posedge clk)
PEnable <= 1'b0;


// Check the result
if (top.mem.memory[16’h50] == 32’h50)
$display("Success");
else
$display("Error, wrong value in memory");
$finish;
end
endmodule
```

**Now it became simple, as we move up.**

```
task write(reg [15:0] addr, reg [31:0] data);

// Drive Control b
@(posedge clk)
PAddr <= addr;
PWData <= data
PWr
PSe

// Toggle Penable
@(posedge clk)
PEnable <= 1'b1
@(posedge clk)
PEnable <= 1'b0

endtask
```

```
module test(PAddr, PWrite, PSel, PRData,
Rst, clk);
// Port declarations omitted...

initial begin
memory

// Check the result
if (top.mem.memory[16’h50] == 32’h50)
$display("Success");
else
$display("Error, wrong value in memory");
$finish;
end

endmodule
```

- MOVEUP

This is fundamental principle of methodology

# Back to ⊔ Why UVM?

**Turning Simulation into Verification**

Pure random stimulus would be nonsensical, so adding constraints to make random stimulus legal is an important part of the verification process, and is explicitly supported by SystemVerilog and UVM

**Checkers, Coverage and Constraints (The 3C's)**

***(Functional Correctness, Functional Completeness, Means to reach Coverage Goals).***

Checkers can be implemented using SystemVerilog assertions or using regular procedural code. Assertions can be embedded within the design-under-test, placed on the external interfaces, or can be part of the verification environment.

- UVM provides mechanisms and guidelines for building checkers into the verification environment and for logging reports

SystemVerilog offers two separate mechanisms for functional coverage collection; property-based coverage (cover directives) and sample-based coverage (covergroups).

- Both can be used in an UVM verification environment

SystemVerilog has dedicated language features for expressing constraints.

- UVM goes further by providing mechanisms that allow constraints to be written as part of a test rather then embedded within dedicated verification components. This and other features of UVM facilitate the creating of reusable verification components.

## [contd]

**Tests and Coverage**

The features enumerated in the verification plan should be captured as a set of coverage statements that together form an executable coverage model.

- The verification plan itself is not part of UVM proper, but is a vital element in the verification process.

UVM provides guidance on how to collect coverage data in a reusable manner.

**Engineering Effort**

Automated coverage collection gives accurate feedback on the progress of the verification effort, and the emphasis on verification planning ensures that resources are focussed on achieving agreed goals

**Verification Reuse**

Verification reuse is enabled by having a modular verification environment where each component has clearly defined responsibilities, by allowing flexibility in the way in which components are configured and used, by having a mechanism to allow imported components to be customized to the application at hand, and by having well-defined coding guidelines to ensure consistency.

- The architecture of UVM has been designed to encourage modular and layered verification environments, where verification components at all layers can be reused in different environments
- Test scenarios can be reused from application to application
- Flexibility is built into the UVM class library

Worlds simplest spec and lets derive a verification plan for it.

**Given condition *X*, the design shall exhibit a**

**be**

It is important for the verification plan to document the verification requirements. Generally speaking, three categories of requirements exist:

stimulus, checking, and coverage

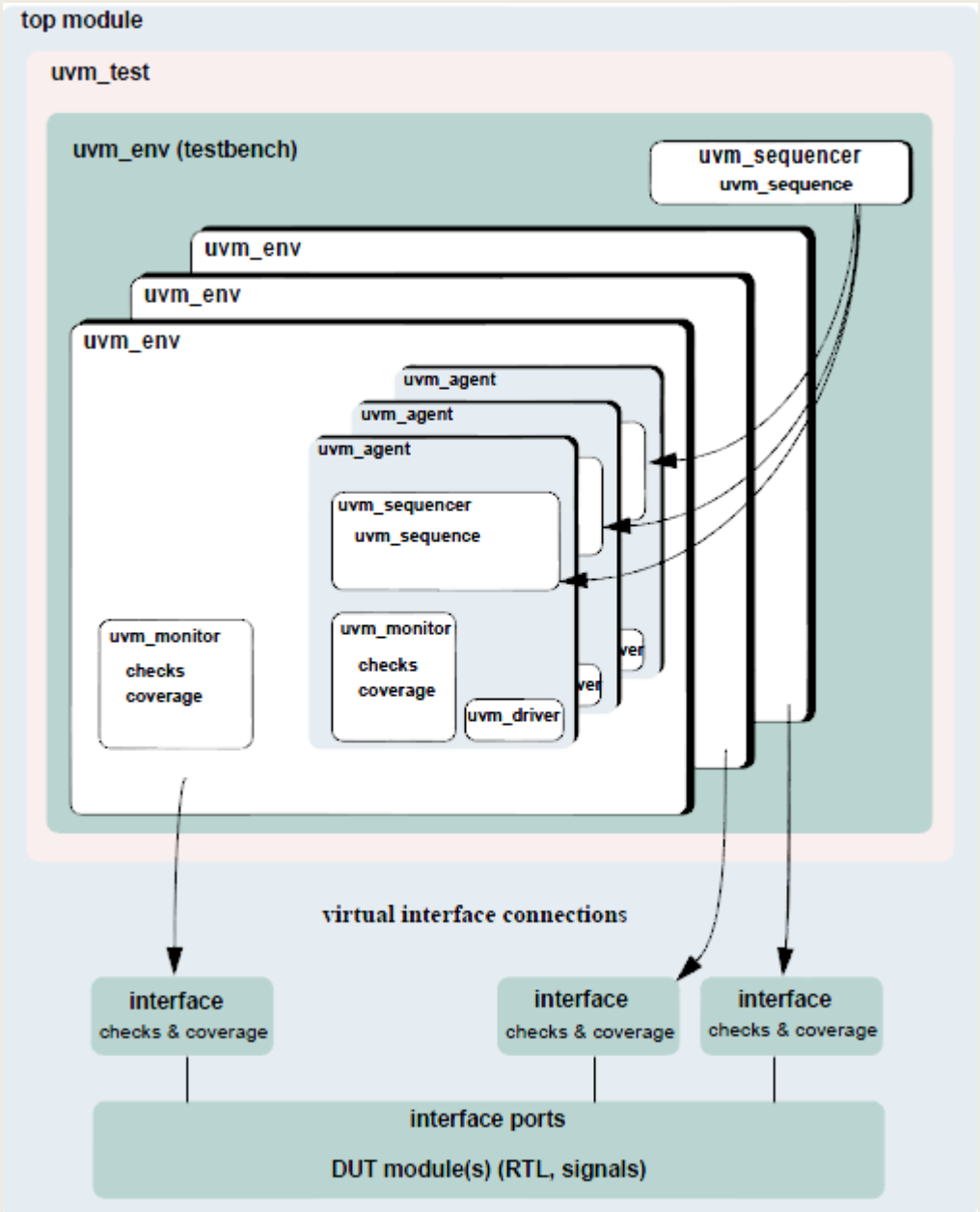
**Stimulus**
*X* is a stimulus the testbench/tests must be able to create

**Checking**
if *X* is the condition given to the design but Y is not the actual behavior of the design, this is an identifiable error symptom that exists in the checker component of the testbench

**Coverage**
the existence of condition *X* must be observed during the course of the simulation, dynamic or static

# The Verification Environment

| DUT | TEST (IN A TOP MODULE) | ENV |
| --- | --- | --- |
| **Design-under-test (or DUT)**<br><br>**Interface to the design-under-test** | **Top-level module**<br>- Instantiation of interface<br>- Instantiation of design-under-test<br>- Test, which instantiates the verification environment<br>- Process to run the test | **Verification environment (or test bench)**<br>- Transaction<br>- Sequencer (stimulus generator)<br>- Driver<br>- Top-level of verification environment<br>- Instantiation of sequencer<br>- Instantiation of driver |


top module
uvm_test
uvm_env (testbench)
uvm_sequencer
uvm_sequence
uvm_env
uvm_env
uvm_env
uvm_agent
uvm_agent
uvm_agent
uvm_sequencer
uvm_sequence
uvm_monitor
checks
coverage
uvm_monitor
checks
coverage
uvm_driver
virtual interface connections
interface
checks & coverage
interface
checks & coverage
interface
checks & coverage
interface ports
DUT module(s) (RTL, signals)

# Data Item (Transaction)

- Data items represent the input to the device under test (DUT). Examples include networking packets, bus transactions, and instructions. The fields and attributes of a data item are derived from the data item's specification.

- For example, the Ethernet protocol specification defines valid values and attributes for an Ethernet data packet.

- In a typical test, many data items are generated and sent to the DUT. By intelligently randomizing data item fields using SystemVerilog constraints, you can create a large number of meaningful tests and maximize coverage.

## Basic UVM flow

- Derive data items from **uvm_sequence_item** so they can be generated in a procedural sequence.
- Add constraints to a da
- Us
au
implement functions such as copy(), compare(), print(), pack(), and so on. Use of these macros is optional, but recommended.
- Define a constructor for the data item.

```
class simple_item extends
uvm_sequence_item;

rand int unsigned addr;
rand int unsigned data;
rand int unsigned delay;

constraint c1 { addr < 16'h2000; }
                                1000; }

                                for general

 `uvm_object_utils_begin(simple_item)
 `uvm_field_int(addr, UVM_ALL_ON)
 `uvm_field_int(data, UVM_ALL_ON)
 `uvm_field_int(delay, UVM_ALL_ON)
 `uvm_object_utils_end

// Constructor
 function new (string name =
"simple_item");
 super.new(name);
 endfunction : new
 endclass : simple_item
```

**Note:**
**You can readjust data item generation for more constraints, add control fields (or Knobs). Refer User Guide 4.1.1, 4.1.2**

**Data Item (Transaction)**
Driver (BFM)
Sequencer
Enabling Scenario Creation
Monitor
Agent
Environment

# Driver (BFM)

- A driver is an active entity that emulates logic that drives the DUT.
- A typical driver repeatedly receives a data item and drives it to the DUT by sampling and driving the DUT signals. (If you have created a verification environment in the past, you probably have implemented driver functionality.)
- For example, a driver controls the read/write signal, address bus, and data bus for a number of clocks cycles to perform a write transfer.

# Basic UVM flow

- Derive the driver.
- Add UVM infrastructure macro.
- Get the resource that defines the virtual interface
- Call get_next_item() to get the next data item for execution from the sequencer.
- Signal the sequencer that the execution of the current data item is done.
- Add your application-specific logic here to execute the data item.

```
class simple_driver extends uvm_driver #(simple_item);

simple_item s_item;
virtual dut_if vif;

 // UVM automation macros for general components
`uvm_component_utils(simple_driver)

// Constructor
 function new (string name = "simple_driver", uvm_component parent);
super.new(name, parent);
endfunction : new

function void build_phase(uvm_phase phase);
 string inst_name;
 super.build_phase(phase);
 if(!uvm_config_db#(virtual dut_if)::get(this, “”,"vif",vif))
 `uvm_fatal("NOVIF", {"virtual interface must be set for: ",
get_full_name(),".vif"});
 endfunction : build_phase

task run_phase(uvm_phase phase);
 forever begin
 // Get the next data item from sequencer (may block).
 seq_item_port.get_next_item(s_item);
 // Execute the item.
 drive_item(s_item);
 seq_item_port.item_done(); // Consume the request.
 end
 endtask : run

 task drive_item (input simple_item item);
 ... // Add your logic here.
 endtask : drive_item
endclass : simple_driver
```

Data Item (Transaction)
**Driver (BFM)**
Sequencer
Enabling Scenario Creation
Monitor
Agent
Environment

# Sequencer

A sequencer is an advanced stimulus generator that controls the item provided to the driver for execution. By default, a sequencer behaves similarly to a simple stimulus generator and returns a random data item upon request from the driver.

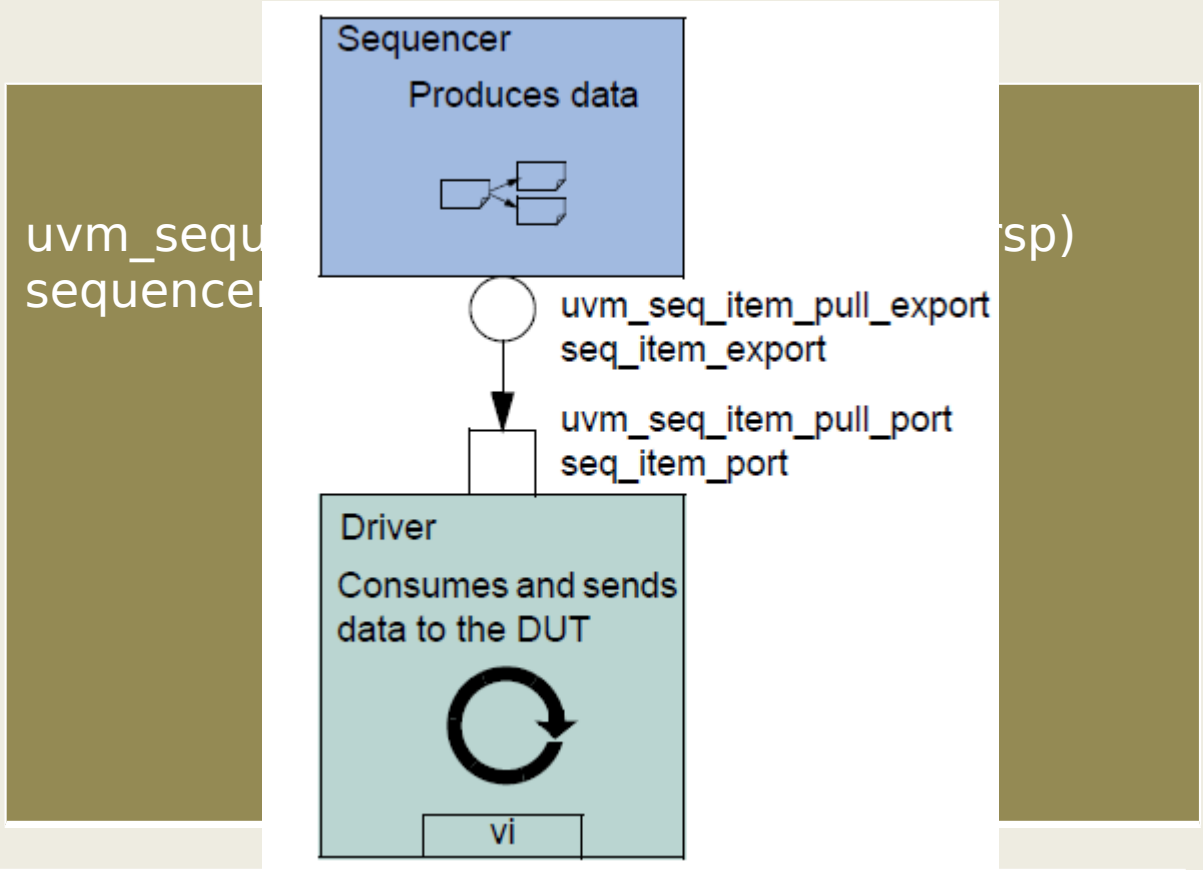
This default behavior allows you to add constraints to the data item class in order to control the distribution of randomized values. Unlike generators that randomize arrays of transactions or one transaction at a time, a sequencer captures important randomization requirements out-of the-box.

**A partial list of the sequencer's built-in capabilities includes:**

- Ability to react to the current state of the DUT for every data item generated.
- Captures the order between data items in user-defined sequences, which forms a more structured and meaningful stimulus pattern.
- Enables time modeling in reusable scenarios.
- Supports declarative and procedural constraints for the same scenario.
- Allows system-level synchronization and control of multiple interfaces.

# Basic UVM flow

- The UVM Class Library provides the **uvm_sequencer** base class, which is parameterized by the request and response item types.
- The **uvm_sequencer** base class contains all of the base functionality required to allow a sequence to communicate with a driver.
- In the class definition, by d
the response t
the same as th
request type.




**Note:**
The only time it is necessary to extend the **uvm_sequencer** class is if you need to add additional functionality, such as additional ports

# Enabling Scenario Creation

The environment user will need to create many test scenarios to verify a given DUT.

Since the verification component developer is usually more familiar with the DUT's protocol, the developer should facilitate the test writing (done by the verification component's user) by doing the following:

- Place knobs in the data item class to simplify declarative test control.
- — Create a library of interesting reusable sequences.
- The environment user controls the environment-generated patterns configuring its sequencers. The user can:
- — Add a sequence of transactions to a sequencer.
- — Modify the sequencer to use specific sequences more often than others.
- — Override the sequencer's main loop to start with a user-defined sequence instead

Data Item (Transaction)
Driver (BFM)
Sequencer
**Enabling Scenario Creation**
Monitor

# Basic UVM flow for a user defined sequence

a) Derive a sequence from t
**uvm_sequence** base class
and specify the request and
respor
type p

b) Use
macro
seque
factor

c) If th
acces
specif
associ
**'uvm_**
macro
seque

d) Imp
body t
scenar
sequence to execute. In
the body task, you can
execute data items and other
sequences using **`uvm_do**

```
class simple_seq_do extends uvm_sequence
#(simple_item);
```

**You can Configure the Sequencer's Default Sequence also. Refer 4.10.3**

```
constraint c1 { count >0; count <50; }
```

**Note:**

The **`uvm_do** macro and all related macros provide a convenient set of calls to create, randomize, and send transaction items in a sequence.

The **`uvm_do** macro delays randomization of the item until the driver has
signaled that it is ready to receive it and the pre_do method has been executed.

Other macro variations allow constraints to be applied to the randomization (uvm_do_with) or bypass the randomization altogether. Refer Figure16 for more details

```
endtask : body
endclass : simple_seq_do
```

**Data Item (Transaction)**
**Driver (BFM)**
**Sequencer**
**Enabling Scenario Creation**
**Monitor**

# Monitor

A monitor is a passive entity that samples DUT signals but does not drive them. Monitors collect coverage information and perform checking. Even though reusable drivers and sequencers drive the bus traffic, they are not used for coverage and checking. Monitors are used instead.

**A monitor:**

— Collects transactions (data items). A monitor extracts signal information from a bus and translates the information into a transaction that can be made available to other components and to the test

**Note:**
**A bus monitor handles all the signals and transactions on a bus, while an agent monitor handles only signals and transactions relevant to a specific agent.**

**Typically, drivers and monitors are built as separate entities (even though they may use the same signals) so they can work independently of each other.**

**However, you can reuse code that is common between a driver**
**and a monitor to save time.**

**Do not have monitors depend on drivers for information so**

# Basic UVM flow

— The monitor collects bus information through a virtual interface (xmi).

— The collected data is used in coverage collection and checking.

— The collected data is exported on an analysis port (**item_collected_port**).

```
class master_monitor extends uvm_monitor;
        virtual bus_if xmi; // SystemVerilog virtual interface
bit checks_enable = 1; // Control checking in monitor and interface
bit coverage_enable = 1; // Control coverage in monitor and
uvm_analysis_port #(simple_item) item_collected_port;
event cov_transaction; // Events needed to trigger covergro
protected simple_item trans_collected;
`uvm_component_utils_begin(master_monitor)
`uvm_field_int(checks_enable, UVM_ALL_ON)
`uvm_field_int(coverage_enable, UVM_ALL_ON)
`uvm_component_utils_end
covergroup cov_trans @cov_transaction;
option.per_instance = 1;
... // Coverage bins definition
endgroup : cov_trans
function new (string name, uvm_component parent);
super.new(name, parent);
cov_trans = new();
cov_trans.set_inst_name({get_full_name(), ".cov_trans"});
trans_collected = new();
item_collected_port = new("item_collected_port", this);
endfunction : new
virtual task run_phase(uvm_phase phase);
fork
collect_transactions(); // Spawn collector task.
join
endtask : run
virtual protected task collect_transactions();
forever begin
@(posedge xmi.sig_clock);
...// Collect the data from the bus into trans_collected.
if (checks_enable)
perform_transfer_checks();
if (coverage_enable)
perform_transfer_coverage();
item_collected_port.write(trans_collected);
end
endtask : collect_transactions
virtual protected function void perform_transfer_coverage();
-> cov_transaction;
endfunction : perform_transfer_coverage
virtual protected function void perform_transfer_checks();
... // Perform data checks on trans_collected.
endfunction : perform_transfer_checks
endclass : master_monitor
```

**Note:**

Coverage collection and checking are conditional because they can affect simulation run-time performance.

If not needed, they can be turned off by setting coverage_enable or checks_enable to 0, using the configuration mechanism. For example:

uvm_config_db#(bit)::set(this,“*.master0.monitor”, “checks_enable”, 0);

Data Item (Transaction)
Driver (BFM)
Sequencer
Enabling Scenario Creation
**Monitor**
Agent

# General understanding of Coverage Definition and Implementation

Three elements of information must be defined for each coverage item.

WHAT to observe

WHEN to observe it

WHERE to observe it

The WHAT, WHEN, and WHERE are embedded **right inside the covergroup code.**

WHAT: The use of coverpoints, user-defined bins, illegal and ignored bins, as well as cross coverage determine the data values, ranges, and sequences the covergroup is tracking.

WHEN: The combination of coverage_event, iff expressions, and the calling of the sample() method determines the sampling frequency.

WHERE: The instantiation of classes with covergroups determines where the coverage information is to be measured.

# One Small feature example

| SUBFEATURE | TYPE | DESCRIPTION |
|---|---|---|
| **A transfer cycle is valid only when cyc_i and stb_i are asserted; otherwise, it is ignored** | Cover (property) | WHAT: {cyc_i, stb_i} = {00, 01, 10, 11}<br>WHEN: each time {cyc_i, stb_i} has a value change<br>WHERE: bus interface on design |

```
…//snippet
wire [1:0] wb_cyc_stb = {cyc_i, stb_i};
reg [1:0] wb_cyc_stb_r;
event ev_cyc_stb_changed;

always @ (posedge clk) begin
    wb_cyc_stb_r <= wb_cyc_stb;
    if (wb_cyc_stb != wb_cyc_stb_r) -> ev_cyc_stb_changed;
end

covergroup c_values @ ev_cyc_stb_changed;
   coverpoint wb_cyc_stb_r;
endgroup: c_values
```

# Agent

- Sequencers, drivers, and monitors can be reused independently, but this requires the environment integrator to learn the names, roles, configuration, and hookup of each of these entities.
- To reduce the amount of work and knowledge required by the test writer, UVM recommends that environment developers create a more abstract container called an agent. Agents can emulate and verify DUT devices.
- They encapsulate a driver, sequencer, and monitor.
- Verification components can contain more than one agent.
- Some agents (for example, master or transmit agents) initiate transactions to the DUT, while other agents (slave or receive agents) react to transaction requests.
- Agents should be configurable so that they can be either active or passive (so 2 Operating Modes for Agents).
- Active agents emulate devices and drive transactions according to test directives. Passive agents only monitor DUT act

Data Item (Transaction)
Driver (BFM)
Sequencer
Enabling Scenario Creation
Monitor

# Basic UVM flow

- The monitor is created using create().
- Write an if condition tests the **is_active** property to determine whether the driver and sequencer are created in this agent. If the agent is set to active (**is_active = UVM_ACTIVE**), the driver and sequencer are created using additional create()calls.
- Both the sequencer and the driver follow the same creation pattern as the monitor.
- This example shows the **is_active** flag as a configuration property for the agent.
- At the environment level, this could be a **num_masters** integer, a **num_slaves** integer, or a **has_bus_monitor** flag.
- Write an if condition that checks to see if the agent is active and, if so, the connection between the sequencer and driver is made using connect().

```
class  simple_agent extends uvm_agent;
   uvm_active_passive_enum is_active;
... // Constructor and UVM automation macros

   uvm_sequencer #(simple_item) sequencer;

   simple_driver driver;
   simple_monitor monitor;

 // Use build() phase to create agents's subcomponents.
     virtual function void build_phase(uvm_phase phase);
     super.build_phase(phase)
 monitor = simple_monitor::type_id::create("monitor",this);
 if (is_active == UVM_ACTIVE) begin

// Build the sequencer and driver.
 sequencer =

uvm_sequencer#(simple_item)::type_id::create("sequencer",this);
 driver = simple_driver::type_id::create("driver",this);
 end
 endfunction : build_phase

virtual function void connect_phase(uvm_phase phase);
 if(is_active == UVM_ACTIVE) begin
 driver.seq_item_port.connect(sequencer.seq_item
 end
 endfunction : connect_phase
 endclass : simple_agent
```

# Environment

- The environment (env) is the top-level component of the verification component. It contains one or more agents, as well as other components such as a bus monitor. The env contains configuration properties that enable you to customize the topology and behavior and make it reusable. For example, active agents can be changed into passive agents when the verification environment is reused in system verification.

- A verification component may contain an environment-level monitor. This bus-level monitor performs checking and coverage for activities that are not necessarily related to a single agent. An agent's monitors can leverage data and events collected by the global monitor.

- The environment class (**uvm_env**) is architected to provide a flexible, reusable, and extendable verification component. The main function of the environment class is to model behavior by generating constrained random traffic, monitoring DUT responses, checking the validity of the protocol activity, and collecting coverage.



# Basic UVM flow

configure all of subcomponents.

- Most verification reuse occurs at the environment level where the user instantiates an environment class and configures it and its agents for specific verification tasks.

- For example, a user might need to change the number of masters and slaves in a new environment as shown below

```
class ahb_env extends uvm_env;
int num_masters;
ahb_master_agent masters[];

`uvm_component_utils_begin(ahb_env)
`uvm_field_int(num_masters, UVM_ALL_ON)
`uvm_component_utils_end

virtual function void build_phase(phase);
string inst_name;
  super.build_phase(phase);
        if(!uvm_config_db#(virtual ubus_if)::get(this,"",
"vif",vif))
      `uvm_fatal("NOVIF",{"virtual interface must be set for: ",
get_full_name(),".vif"});
masters = new[num_masters];
     for(int i = 0; i < num_masters; i++) begin
           $sformat(inst_name, "masters[%0d]", i);
           masters[i] =
ahb_master_agent::type_id::create(inst_name,this);
     end
// Build slaves and other components.

endfunction

function new(string name, uvm_component pare
       super.new(name, parent);
  endfunction : new

endclass
```

Data Item (Transaction)
Driver (BFM)
Sequencer
Enabling Scenario Creation
Monitor
Agent

# [Info] Most noticeable recent additions

- An end-of-test objection mechanism to ease the task of cleaning up at the end of a verification run
- A callback mechanism that provides an alternative to the factory for customizing behavior
- A report catcher to ease the task of customized report handling
- A *heartbeat* mechanisms to monitor the liveness of verification components.

**Here is a list of the changes required to convert from UVM-EA to fully compliant UVM-1.0 code**
**(UVM-1.0 was released in February 2011)**

- Rename the standard phase methods by appending **_phase** to each name
- Add the argument **(uvm_phase phase)** to each of the standard phase methods
- Don't forget to change the calls to **super.build_phase(phase);**
- Replace the old sequencer macro **uvm_sequencer_utils** with **uvm_component_utils**
- Do not use the deprecated **uvm_update_sequence_lib(_and_item)**. Start sequences with **start**
- Replace calls to **set/get_config_int/string/object** with the new **uvm_config_db** or **uvm_resource_db**
- Replace **set_global_timeout** with calls to **raise_objection** and **drop_objection**
- Replace **uvm_top.stop_timeout**, which no longer exists
- When using **uvm_class_comparator**, override **do_compare** instead of **comp**

## How about ending this *Topic1: World of UVM* with a small interesting piece of

| Component | Transaction |
| --- | --- |
| extends uvm_component<br><br>contains<br>`uvm_component_utils<br><br>may define ports and exports<br><br>contains<br>function new(string name,<br>uvm_component parent);<br>super.new(name,<br>parent);<br>endfunction<br><br>may define phase methods<br>build, connect, run, ... | extends uvm_sequence_item<br><br>contains `uvm_object_utils<br><br>(no ports or exports)<br><br>contains<br>function new (string name =<br>"");<br>super.new(name);<br><br>endfunction<br><br>may define utility methods<br>convert2string, do_copy,<br>do_compare |

1. World of UVM
2. Brief understanding of Migration from verilog to UVM
3. Phases
4. Defining a Skelton (walk through a code)
5. Sample Make file and run an example

# 2. Brief understanding of Migration from a HDL to UVM

We will see...

a. **Modules to the UVM Component**
b. **HDL Processes to the UVM Run Phase**
c. **HDL Input and Output Ports to TLM Ports and Exports**
d. **Verilog Parameters to UVM Configurations**
e. **VHDL Records to Transactions**

# 2.a. Modules to the UVM Component

```
// VERILOG
module A;
  wire w;
  B B_instance( .p(w) );
  C C_instance( .q(w) );
endmodule
```

```
// UVM
class A extends uvm_component;

  `uvm_component_utils(A)

  B B_h;
  C C_h;

//constructor
  function new(string name, uvm_component parent);
    super.new(name, parent);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    B_h = B::type_id::create("B_h", this); //factory section
7.2.1
    C_h = C::type_id::create("C_h", this);
  endfunction

  function void connect_phase(uvm_phase phase);
    B_h.p_port.connect( C_h.q_export );
  endfunction

endclass
```

# [contd] explanation for previous slide

- In the example above, a top-level block named A instantiates two lower level blocks named B and C.
- The Verilog module instances named B_instance and C_instance are replaced in UVM by the SystemVerilog variables B_h and C_h, the suffix _h indicating that the variable is a *handle* to an object rather than being the object itself.
- The UVM equivalent of Verilog module instantiation is done within function **build_phase**, and the UVM equivalent of Verilog port connection is done within function **connect_phase**.
- UVM ports are analogous to Verilog ports and can even be connected by name like Verilog ports, although unlike Verilog ports UVM ports are used to pass transactions using function calls (more on this later).
- The rest of the UVM component should be treated like boilerplate code and just reproduced as shown in each and every component. This includes the uvm_component_utils line, the function **new** (which is known technically as the constructor of the class), and the skeleton of the functions **build_phase** and **connect_phase**.

# 2.b HDL Processes to the UVM Run Phase

```
-- VHDL
entity D is
end entity;

architecture V1 of D is
begin
  P1: process
  begin
    report "Hello world 1";
    wait;
  end process;

  P2: process
  begin
    report "Hello world 2";
    wait;
  end process;
end architecture;
```

```
// UVM
class D extends uvm_component;

  `uvm_component_utils(D)

  function new(string name, uvm_component
parent);
    super.new(name, parent);
  endfunction

  task run;
    fork
      begin: P1
        `uvm_info("", "Hello world 1",
UVM_NONE)
      end
      begin: P2
        `uvm_info("", "Hello world 2",
UVM_NONE)
      end
    join
  endtask

endclass
```

**Any code written within the run method, which must be a SystemVerilog task, executes as a process during simulation.**

**A UVM component can only have a single run method, so if you want multiple concurrent processes within a single UVM component, you have to spawn them off using fork-join.**

**Note that there are no component instances**

# 2.c. TLM

```
module producer (my_port);
  output [31:0] my_port;
  reg    [31:0] my_port;
  initial
    my_port = 99;
endmodule

module consumer (my_export);
  input [31:0] my_export;
  always @(my_export)
    $display("my_export = %0d",
my_export);
endmodule

module top;
  wire [31:0] w;
  producer producer_inst(
.my_port(w) );
  consumer consumer_inst(
.my_export(w) );
endmodule
```

```
// UVM
class producer extends uvm_component;
  `uvm_component_utils(producer)

  uvm_blocking_put_port #(int) my_port;

  function new(string name, uvm_component parent);
    super.new(name, parent);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    my_port = new("my_port", this);
  endfunction

  task run_phase(uvm_phase phase);
    my_port.put(99);
  endtask
endclass


class consumer extends uvm_component;
  `uvm_component_utils(consumer)

  uvm_blocking_put_imp #(int, consumer) my_export;

  function new(string name, uvm_component parent);
    super.new(name, parent);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    my_export = new("my_export", this);
  endfunction

  task put(int arg);
    `uvm_info("", $sformatf("Called put(%0d)", arg), UVM_NONE)
  endtask
endclass
```

## contd

```
class top extends uvm_component;
  `uvm_component_utils(top)

  producer producer_h;
  consumer consumer_h;

  function new(string name, uvm_component
parent);
    super.new(name, parent);
  endfunction

  function void build_phase(uvm_phase
phase);
    super.build_phase(phase);
    producer_h =
producer::type_id::create("producer_h",
this);
    consumer_h =
consumer::type_id::create("consumer_h",
this);
  endfunction

  function void connect_phase(uvm_phase
phase);

producer_h.my_port.connect( consumer_h.my
_export );
  endfunction
endclass
```

# [contd] Explanation of previous 2 slides

- Whereas in Verilog a producer would make an assignment to an output port, in UVM a producer would make a call to the **put** method through a port.

- Whereas in Verilog a consumer would have a process sensitive to an input port and would then read the value of that input port when it changes, in UVM a consumer would provide an implementation of the **put** method and also an **export** that can be hooked to the corresponding **port** on the producer.
- In the example above you can see the producer making the call
  my_port.put(99);
  and the consumer providing an implementation of put
  function void put(int arg);.

  In order to connect the two together, the producer declares a port
  uvm_blocking_put_port #(int) my_port;
  the consumer declares an export
  uvm_blocking_put_imp #(int, consumer) my_export;
  and the top-level component makes the connection between the two
  producer_h.my_port.connect( consumer_h.my_export );.
- Transaction-level communication using methods such as **put** does not involve wires (Verilog) or signals (VHDL), nor does it necessarily involve events or synchronization with the simulation kernel, but nonetheless it does have the effect of passing messages between components in the UVM test bench.

# 2.d Verilog Parameters to UVM Configurations

**SystemVerilog provides the convenience of allowing the parameters to be defined on the module line and overridden on the module instance line:**

```
// SystemVerilog
module producer #(parameter bit param1 = 0,
int param2 = 0, string param3 = "");
  ...
endmodule

module top;
  producer #( .param1(1'b1), .param2(2),
.param3("3") ) producer_inst ();
endmodule
```

## [First…Summary of how its done in UVM]

**One UVM configuration object for each component that needs to be parameterized**

**Define a class that represents a parameter block and contains all the parameters for a given component.**

**The class xtends uvm_object rather than uvm_sequence_item.**

**It is recommended to define the utility method convert2string, because it can be handy for debugging**

***..UVM coding snippet in next slides (right half)***

# contd

```
// UVM
class producer_config extends uvm_object;

   // Standard macro for a config object, transaction, or sequence
  `uvm_object_utils(producer_config)

  rand bit param1;
  rand int param2;
  string   param3;
  // Other configuration parameters

  // Standard constructor for a config object, transaction, or sequence
  function new (string name = "");
    super.new(name);
  endfunction

  // Standard utility method for a config object or transaction
  function string convert2string;
    return $sformatf("param1=%b, param2=%0d, param3=%s", param1,
param2, param3);
  endfunction

endclass
```

# contd

```
// UVM
class producer extends uvm_component;
  ...
  producer_config config_h;

  // Configuration parameters
  bit    param1 = 0;
  int    param2 = 0;
  string param3;
  ...
  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    my_port = new("my_port", this);

    begin
      if ( uvm_config_db #(producer_config)::get(this, "", "config",
config_h) )
      begin
        param1 = config_h.param1;  // Local parameters copied from
configuration object
        param2 = config_h.param2;
        param3 = config_h.param3;
      end
    end
  endfunction
  ...
endclass
```

# contd

```
// UVM
class my_test extends uvm_test;
  ...
  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    begin
      producer_config config_h = new;
      config_h.param1 = 1;  // Set test-specific values for
configuration parameters
      config_h.param2 = 2;
      config_h.param3 = 3;
      uvm_config_db #(my_agent_config)::set(this, "*.*producer*",
"config", config_h);
    end
    top_h = top::type_id::create("top_h", this);
  endfunction
endclass
```

# 2.e VHDL Records to UVM Transactions

```
-- VHDL
package my_pkg
is
  type
my_transaction is
    record
      data: integer;
    end record;
end package;
```

```
// UVM
package my_pkg;
  class my_transaction extends
uvm_sequence_item;

    `uvm_object_utils(my_transaction)

    rand int data;
    // Other attributes of the transaction go
here...

    function new (string name = "");
      super.new(name);
    endfunction

  endclass: my_transaction
Endpackage
```

1. World of UVM
2. Brief understanding of Migration from verilog to UVM
3. Phases
4. Defining a Skelton (walk through a simple code)
5. Sample Make file and run an example

# 3. Phases

**In Conventional Verilog there are three fixed phases**

1. **Compilation (or analysis),**
2. **Elaboration, and**
3. **Simulation**

**Because UVM *is* SystemVerilog it shares SystemVerilog's phases, but the SystemVerilog simulation phase is further broken down within UVM into:**

**new**
**build_phase**
**connect_phase**
**end_of_elaboration_phase**
**start_of_simulation_phase**
**run_phase**
- **reset_phase**
- **configure_phase**
- **main_phase**
- **shutdown_phase**

**extract_phase**
**check_phase**
**report_phase**
**final_phase**

**Details in Next Slide**



PPT Notes Added

## [contd]

build()

UVM provides an **objection mechanism** to allow hierarchical status communication among components.

There is a **built-in objection for each phase**, which provides a way for components and objects to synchronize their testing activity and indicate when it is safe to end the phase and, ultimately, the test.

In general, the process is for a component or sequence to raise a phase objection at the beginning of an activity that must be completed before the phase stops and to drop the objection at the end of that activity.
Once all of the raised objections are dropped, the phase terminates.

In simulation, agents may have a meaningful agenda to be achieved before the test goals can be declared as done. For example, a master agent may need to complete all its read and write operations before the run phase should be allowed to stop. A reactive slave agent may not object to the end of test

```
class interesting_sequence extends
uvm_sequence#(data_item);

task pre_body();
// raise objection if started as a root
sequence
starting_phase.raise_objection(this);
endtask

task body();
//do interesting activity
...
endtask

task post_body();
// drop objection if started as a root
sequence
starting_phase.drop_objection(this);
endtask
endclass
```

# Sample Log to interpret Phases

```
uvm_test_top.t_env [uvm_test_top.t_env] Build
uvm_test_top.t_env.ag1 [uvm_test_top.t_env.ag1] Build
uvm_test_top.t_env.ag1.drv [uvm_test_top.t_env.ag1.drv] Build
uvm_test_top.t_env.ag1.mon [uvm_test_top.t_env.ag1.mon] Build
uvm_test_top.t_env.ag2 [uvm_test_top.t_env.ag2] Build
uvm_test_top.t_env.ag2.drv [uvm_test_top.t_env.ag2.drv] Build
uvm_test_top.t_env.ag2.mon [uvm_test_top.t_env.ag2.mon] Build
uvm_test_top.t_env.ag1.drv [uvm_test_top.t_env.ag1.drv] Connect
uvm_test_top.t_env.ag1.mon [uvm_test_top.t_env.ag1.mon] Connect
uvm_test_top.t_env.ag1 [uvm_test_top.t_env.ag1] Connect
uvm_test_top.t_env.ag2.drv [uvm_test_top.t_env.ag2.drv] Connect
uvm_test_top.t_env.ag2.mon [uvm_test_top.t_env.ag2.mon] Connect
uvm_test_top.t_env.ag2 [uvm_test_top.t_env.ag2] Connect
uvm_test_top.t_env [uvm_test_top.t_env] Connect
uvm_test_top.t_env.ag1.drv [uvm_test_top.t_env.ag1.drv] End_of_elaboration
uvm_test_top.t_env.ag1.mon [uvm_test_top.t_env.ag1.mon] End_of_elaboration
uvm_test_top.t_env.ag1 [uvm_test_top.t_env.ag1] End_of_elaboration
uvm_test_top.t_env.ag2.drv [uvm_test_top.t_env.ag2.drv] End_of_elaboration
uvm_test_top.t_env.ag2.mon [uvm_test_top.t_env.ag2.mon] End_of_elaboration
uvm_test_top.t_env.ag2 [uvm_test_top.t_env.ag2] End_of_elaboration
uvm_test_top.t_env [uvm_test_top.t_env] End_of_elaboration
uvm_test_top [uvm_test_top] End_of_elaboration
----------------------------------------------------------------------
Name                    Type              Size              Value
----------------------------------------------------------------------
uvm_test_top            test1             -         uvm_test_top@2
  t_env                 env               -           t_env@4
    ag1                 agent             -             ag1@6
      drv               driver            -           drv@12
        rsp_port        uvm_analysis_port -        rsp_port@16
        sqr_pull_port   uvm_seq_item_pull_+ -      sqr_pull_port@14
      mon               monitor           -           mon@10
    ag2                 agent             -           ag2@8
      drv               driver            -           drv@20
        rsp_port        uvm_analysis_port -        rsp_port@24
        sqr_pull_port   uvm_seq_item_pull_+ -      sqr_pull_port@22
      mon               monitor           -           mon@18

uvm_test_top.t_env.ag1.drv[uvm_test_top.t_env.ag1.drv]Start_of_simulation
uvm_test_top.t_env.ag1.mon[uvm_test_top.t_env.ag1.mon]Start_of_simulation
uvm_test_top.t_env.ag1[uvm_test_top.t_env.ag1]Start_of_simulation
uvm_test_top.t_env.ag2.drv[uvm_test_top.t_env.ag2.drv]Start_of_simulation
```

1. World of UVM
2. Brief understanding of Migration from verilog to UVM
3. Phases
4. Defining a Skelton (walk through a simple code)
5. Sample Make file and run an example

# 4. UVM Boilerplate Code Framework (like a rough Skelton)

## DUT

```
`include "uvm_macros.svh"

interface dut_if();
 // Example interface: put your pins and modports here

endinterface: dut_if
```

```
modul
  // Ex
your D

  import uvm_pkg::*;


endmodule: dut
```

## TEST (IN A TOP MODULE)

```
module top;
  import uvm_pkg::*;
  import my_pkg::*;

  dut_if dut_if1 ();
  dut    dut1 ( ._if(dut_if1) );

  // Clock generator
  initial
    dut_if1.reset = 1;
    repeat(3) @(negedge
dut_if1.clock);
    dut_if1.reset = 0;
  end
initial
  begin: blk
    uvm_config_db #(virtual
dut_if)::set(null, "*", "dut_vi",
dut_if1);

    uvm_top.enable_print_topology
= 1;
    uvm_top.finish_on_completion
= 1;

    run_test("my_test");
```

## ENV

Can be arranged in two sets of packages (small+big)
***my_seq_library.***

**Full Code Runs out of the Box (any simulator)**
**Here is the Link**

+

**my_pkg**

This library has classes are for
▭

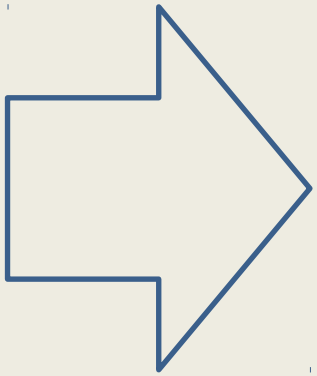
callback interface
driver
monitor
agent configuration
agent
subscriber
env
callback modification
report catcher
test

1. World of UVM
2. Brief understanding of Migration from verilog to UVM
3. Phases
4. Defining a Skelton
5. Sample Make file and run an example

# 5. Sample Make file and a Run Example (Hierarchy View in DVE)

View Main Makefile

View VCS Specific Makefile



Open VNC or NX
For Live demo

[General Ending Note]

# What’s the strategy for a good verification plan ?

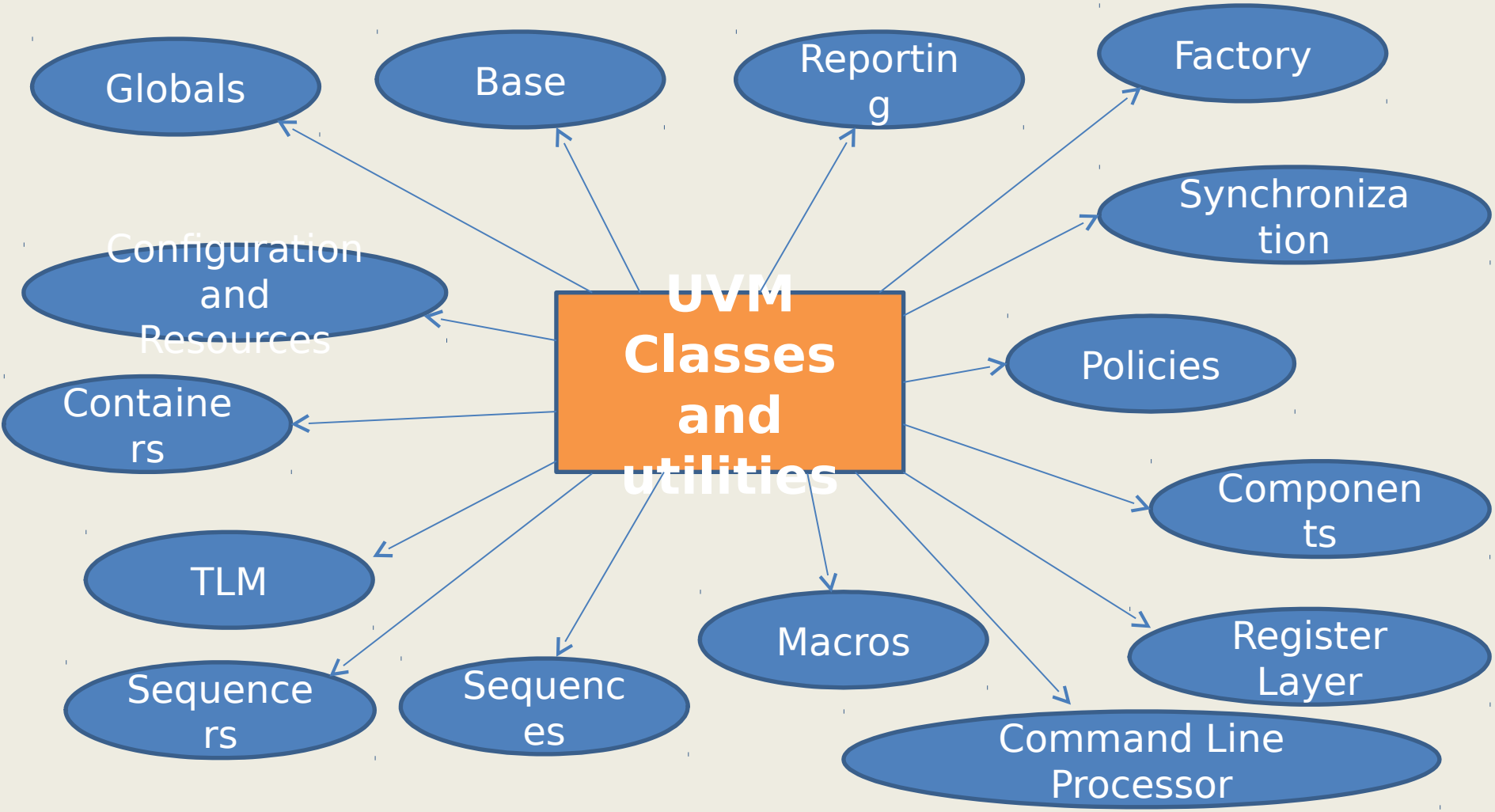
- A good verification methodology starts with a statement of the function the DUT is intended to perform.
- From this is derived a verification plan, broken down feature-by-feature, and agreed in advance by all those with a specific interest in creating a working product. This verification plan is the basis for the whole verification process.
- Verification is only complete when every item on the plan has been tested to an acceptable level, where the meaning of "acceptable" and the priorities assigned to testing the various features have also been agreed in advance and are continually reviewed during the project.
- Verification of complex systems should not be reliant on manual inspection of detailed waveforms and vector sets.
- Functional checking must be automated if the process is to scale well, as must the collection of verification metrics such as the coverage of features in the verification plan and the number of bugs found by each test.

**Take help of UVM.**
**UVM is here to stay. Period.**

# Thankyou

Pls support yourself if you want to getinto the world of UVM.

1. UVM user guide
2. SV P-1800 LRM
3. Verification Academy
4. Vendor support sites (supportnet from MentorGraphics, solvnet of Synopsys, support.cadence.com)
5. Uvmworld.org
6. Accelerra
7. Testbench.in, asicworld, Ubus Example part of User Guide, other sites
8. Legacy blogs of ovm, vmm
9. Google




PPT Notes Added